Sequential assignment:

Recursion, Fractals, and 2D Graphics

Object-Oriented Programming and GUI

May 25, 2026

Theory

A fractal is a geometric object that is difficult to describe using geometric shapes. It has the property that it repeats itself, but on an increasingly smaller scale.

Practice

To create a tree, clouds, a landscape, or a circulatory system, a model is often generated based on parameters selected by the artist. The completed model is used in a game or a movie.

Tips

- Most often, it is enough to use recursion.
- The obtained effects are an interesting graphical element.

References

- Fractal (Wikipedia)
- Sierpiński triangle (Wikipedia)
- Project for reading

I. Recursion exercises

1. Implement the method and mark it with the *TODO 01* tag, implementing recursive addition of two integer numbers.
   (a) Create the `Recursion` class in a new file `Recursion.java`.
   (b) Define the static method `sum(int a, int b)` returning a value of type `int`.
   (c) Implement the base case of the recursion.
      - Check the condition `b == 0`.
      - If the condition is met, return the value `a`.
   (d) Implement the recursive step.

- Return the invocation `sum(++a, -b)`.
- Ensure that `a` is pre-incremented and `b` is pre-decremented before being passed to the invocation.

2. Implement the method and mark it with the *TODO 02* tag, implementing recursive iteration without using loops.

   (a) Define the static method `times(int i)` returning `void` inside the `Recursion` class.

   (b) Implement the base case of the recursion.

   - Check the condition `i == 0`.
   - If the condition is met, terminate the method with the `return` statement.

   (c) Print the current value of the parameter using `System.out.println("i:  " + i)`.

   (d) Invoke `times(-i)` recursively.

   - Pass the value of `i` decreased by 1 before the invocation (pre-decrement).

3. Implement the `main` method and mark it with the *TODO 03* tag, invoking the `sum` and `times` methods.

   (a) Invoke `sum(6, 9)` and print the result using `System.out.println(...)`.

   - Expect the value `15`.

   (b) Invoke `times(10)` to start the recursive iteration.

   - Expect consecutive numbers from `10` to `1` to be printed, each on a separate line.

4. Check whether the result of running the program will be:

```
15
i: 10
i: 9
i: 8
i: 7
i: 6
i: 5
i: 4
i: 3
i: 2
i: 1
```

5. present the result of your work to the instructor

6. Implement the `array_sum` method marked with the *TODO 04* tag, calculating the sum of array elements using recursion.

   (a) Define the static method `array_sum(int[] tab, int i)` returning a value of type `int` inside the `Recursion` class.

   (b) Implement the base case of the recursion.

   - Check the condition `i == 0`.

- If the condition is met, return `tab[0]`.

(c) Implement the recursive step.

- Return the expression `tab[i] + array_sum(tab, i - 1)`.
- Ensure that each invocation adds the current element to the sum of the previous elements.

(d) Implement a test invocation in the `main` method.

- Create the array `int[] numbers = {1, 2, 3, 4, 5}`.
- Invoke `array_sum(numbers, numbers.length - 1)` and print the result.
- Use the format `"Array sum:  " + result`.

7. Check whether the result of running the program will be:

```
15
i: 10
i: 9
i: 8
i: 7
i: 6
i: 5
i: 4
i: 3
i: 2
i: 1
Array sum: 15
```

8. present the result of your work to the instructor

II. Drawing a tree

9. Download the `Main`, `TreePanel`, `SierpinskiPanel`, and `KochPanel` classes from the assignment content.

10. In the `TreeFrame` class, implement the `draw` method marked with the *TODO 05* tag, drawing the left branch of the tree.

(a) Define the method `private void draw(Graphics g, int xFrom, int yFrom, double radius, double angle)`.

(b) Declare the variable `double deltaAngle = 0.25` defining the deviation of the branch angle from the trunk axis.

(c) Calculate the coordinates of the end of the left branch.

- Declare `double leftEndX = xFrom + Math.sin(angle + deltaAngle) * radius`.
- Declare `double leftEndY = yFrom + Math.cos(angle + deltaAngle) * radius`.

(d) Set the drawing color using `g.setColor(Color.BLACK)`.

(e) Draw the line of the left branch using `g.drawLine(xFrom, yFrom, (int) leftEndX, (int) leftEndY)`.

11. In the body of the `TreeFrame` class, implement the `paintComponent` method marked with the *TODO 06* tag, initializing drawing parameters and starting the drawing of the tree.
    (a) Override the method `public void paintComponent(Graphics g)`.
    (b) Invoke `super.paintComponent(g)` as the first statement of the method.
        - Ensure correct background refreshing by the superclass.
    (c) Declare the variable `int xFrom = getWIdth()/2` as the X coordinate of the trunk start point.
    (d) Declare the variable `int yFrom = getHeight() - 20` as the Y coordinate of the trunk start point.
    (e) Declare the variable `int radius = 100` defining the length of the first branch.
    (f) Declare the variable `double angle = Math.PI` as the initial angle pointing upward.
    (g) Invoke the method `draw(g, xFrom, yFrom, radius, angle)`.
12. Implement drawing the right branch marked with the *TODO 07* tag, extending the `draw` method with a symmetrical branch.
    (a) Calculate the coordinates of the end of the right branch inside the `draw` method.
        - Declare `double rightEndX = xFrom + Math.sin(angle - deltaAngle) * radius`.
        - Declare `double rightEndY = yFrom + Math.cos(angle - deltaAngle) * radius`.
        - Note that the angle is subtracted (`angle - deltaAngle`), not added.
    (b) Draw the line of the right branch using `g.drawLine(xFrom, yFrom, (int) rightEndX, (int) rightEndY)`.
13. present the result of your work to the instructor
14. Implement recursion in the `draw` method marked with the *TODO 08* tag, enabling the drawing of a complete multi-level tree.
    (a) Add the parameter `int depth` to the signature of the `draw` method.
        - Update the invocation from the `paintComponet` method – pass the value `10` as the depth.
    (b) Implement the base case at the beginning of the `draw` method.
        - Check the condition `depth == 0`.
        - If the condition is met, terminate the method using `return`.
    (c) Add a recursive invocation of the `draw` method for the left branch after drawing its line.
        - Pass `(int) leftEndX` and `(int) leftEndY` as the new start point.
        - Pass `radius * 0.7` as the shortened length of the next branch.
        - Pass `angle + deltaAngle` as the new angle.
        - Pass `depth - 1` as the decreased depth.

(d) Add a recursive invocation of the `draw` method for the right branch after drawing its line.

- Pass `(int) rightEndX` and `(int) rightEndY` as the new start point.
- Pass `radius * 0.7` as the shortened length of the next branch.
- Pass `angle - deltaAngle` as the new angle.
- Pass `depth - 1` as the decreased depth.

15. Check whether the result of drawing the tree is similar to the following:

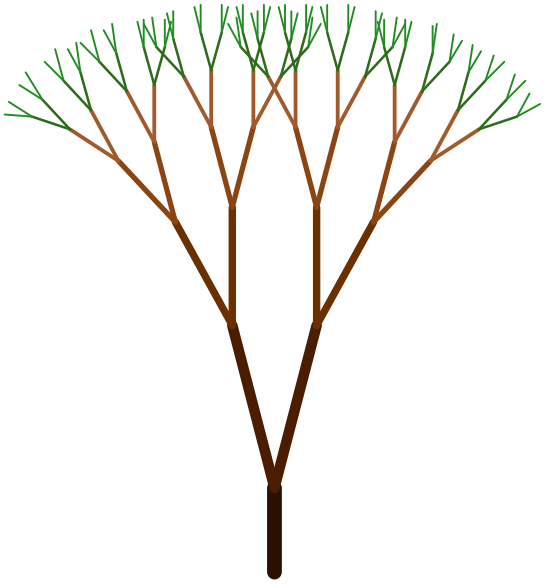


Figure 1: Fractal tree drawn in Swing

16. Implement line thickness differentiation marked with the *TODO 09* tag, giving branches different thicknesses depending on the recursion level.

(a) Modify the `paintComponent` method: cast `g` to `Graphics2D`.

- Add the statement `Graphics2D g2d = (Graphics2D) g`.
- Update the `draw` invocation, passing `g2d` instead of `g`.

(b) Modify the signature of the `draw` method by changing the type of the first parameter to `Graphics2D`.

(c) Set the line thickness at the beginning of the `draw` method.

- Use `g2d.setStroke(new BasicStroke(depth))`.
- Ensure that branches closer to the root (large `depth`) are thicker.
- Add the import `java.awt.BasicStroke`.

17. Implement branch coloring marked with the *TODO 10* tag, changing the line color depending on the recursion depth.

(a) Define the branch color inside the `draw` method, before drawing the line.

- For branches with `depth > 3`, set brown: `new Color(101, 67, 33)`.
- For the remaining ones (leaves), set green: `new Color(34, 139, 34)`.
- Use an `if/else` statement and invoke `g2d.setColor(...)`.

18. Implement random variation of branch angles and lengths marked with the *TODO 11* tag, giving the tree a natural, organic appearance.

    (a) Add the field `private final Random random = new Random()` at the `TreeFrame` class level.

    - Add the import `java.util.Random`.

    (b) Modify the initialization of the `deltaAngle` variable inside the `draw` method.

    - Replace the constant `0.25` with the expression `0.25 + random.nextDouble() * 0.1`.
    - Ensure that each method invocation generates a slightly different branching angle.

    (c) Modify the branch shortening multiplier in both recursive invocations of `draw`.

    - Replace the constant `0.7` with the expression `0.6 + random.nextDouble() * 0.2`.
    - Ensure random branch lengths at each level.

19. Implement background color change marked with the *TODO 12* tag, creating a dark background for the bonsai tree.

    (a) Inside the `paint` method, immediately after invoking `super.paint(g)`, fill the window background with a color.

    - Set the color using `g2d.setColor(new Color(20, 20, 20))`.
    - Fill the entire window area using `g2d.fillRect(0, 0, getWidth(), getHeight())`.
    - Make sure that the `fillRect` statement precedes the invocation of `draw(...)`.

20. 🏁 Milestone 1: present the result of your work to the instructor

21. Implement a program drawing the Sierpiński triangle.

    (a) In the `SierpinskiPanel` class, implement the method `private void drawSierpinski(Graphics g, int[] x, int[] y, int depth)` marked with the *TODO 13* tag.

    i. Implement the base case: when `depth == 0`, draw a filled triangle.

    - Invoke `g.fillPolygon(x, y, 3)`.
    - Terminate the method with the `return` statement.

    ii. Calculate the midpoints of each of the three sides of the triangle.

    - `int mx1 = (x[0] + x[1]) / 2,  int my1 = (y[0] + y[1]) / 2`
    - `int mx2 = (x[1] + x[2]) / 2,  int my2 = (y[1] + y[2]) / 2`
    - `int mx3 = (x[0] + x[2]) / 2,  int my3 = (y[0] + y[2]) / 2`

    iii. Invoke `drawSierpinski` recursively for the top triangle.

    - Pass the X array: `new int[]{x[0], mx1, mx3}`.

- Pass the Y array: `new int[]{y[0], my1, my3}`.
- Pass `depth - 1`.

iv. Invoke `drawSierpinski` recursively for the lower-left triangle.
- Pass the X array: `new int[]{mx1, x[1], mx2}`.
- Pass the Y array: `new int[]{my1, y[1], my2}`.
- Pass `depth - 1`.

v. Invoke `drawSierpinski` recursively for the lower-right triangle.
- Pass the X array: `new int[]{mx3, mx2, x[2]}`.
- Pass the Y array: `new int[]{my3, my2, y[2]}`.
- Pass `depth - 1`.

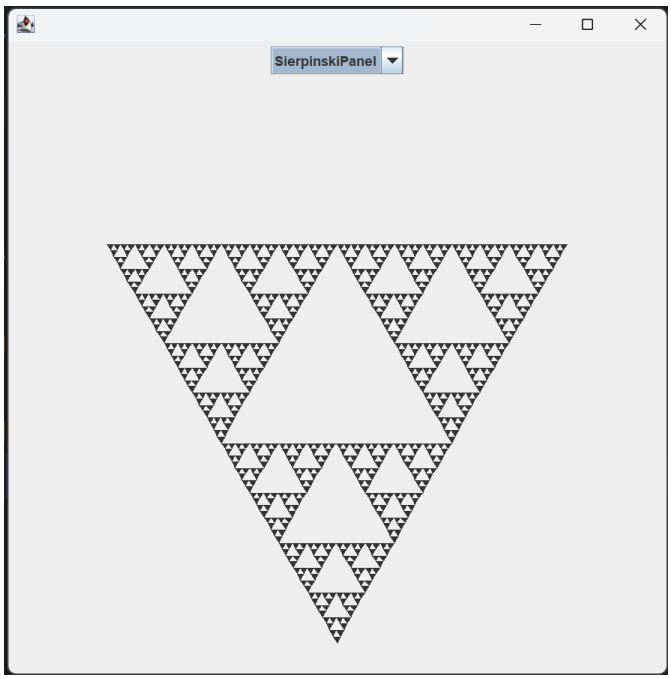
(b) In the `SierpinskiPanel` class, override the method `public void paintComponent(Graphics g)`.

- Invoke `super.paintComponent(g)`.
- Declare two arrays of type `int` named `x` and `y`, each of size `3`.
  i. The `x` array will store the horizontal coordinates of the polygon points.
  ii. The `y` array will store the vertical coordinates of the polygon points.
- Calculate the value of the variable `step` of type `double`.
  i. Determine the full angle of the circle as `2 * Math.PI`.
  ii. Divide the result by `3` to obtain an equal angular spacing between consecutive points.
  iii. Store the result in the variable `step`.
- Calculate the radius of the shape and store it in the variable `r` of type `double`.
  i. Get the current width of the component by invoking `getWidth()`.
  ii. Get the current height of the component by invoking `getHeight()`.
  iii. Choose the smaller of these values by invoking `Math.min(...)`.
  iv. Divide the result by `2.0` to determine a radius that fits inside the component.
  v. Multiply the result by `0.9` to leave a margin from the component edges.
- Create a loop iterating over all indices of the `x` array.
  i. Start the iteration from index `0`.
  ii. Continue the iteration as long as index `i` is less than `x.length`.
  iii. After each iteration, increase the index value by `1`.
- In each iteration, calculate the horizontal coordinate of the point and store it in the `x` array.
  i. Calculate the angle of the current point as `step * i`.
  ii. Calculate the sine of the angle by invoking `Math.sin(...)`.
  iii. Multiply the result by the radius `r`.
  iv. Convert the result to type `int`.
  v. Add half of the component width using `getWidth() / 2` to move the center of the shape to the center of the component.

vi. Store the result in the `x[i]` array.

- In each iteration, calculate the vertical coordinate of the point and store it in the `y` array.
  i. Calculate the angle of the current point as `step * i`.
  ii. Calculate the cosine of the angle by invoking `Math.cos(...)`.
  iii. Multiply the result by the radius `r`.
  iv. Convert the result to type `int`.
  v. Add half of the component height using `getHeight() / 2` to move the center of the shape to the center of the component.
  vi. Store the result in the `y[i]` array.
- Invoke `drawSierpinski(g, x, y, 6)`.

22. Check whether the result of drawing the triangle is similar to the following:



23. ★ Milestone 2: present the result of your work to the instructor

24. The `KochPanel` class, which extends `JPanel`, will be responsible for drawing the Koch snowflake.

    (a) Add a private field `depth` of type `int`, which defines the recursion depth of the fractal.
    - Set the initial value to `5`.

    (b) Declare a private, static inner class `Point`, which stores point coordinates.
    i. Add fields `x` and `y` of type `double`.
    ii. Implement the constructor `Point(double x, double y)`.
    iii. Assign the passed coordinates to the fields `x` and `y`.

    (c) Implement the private method `kochSegment(Point p1, Point p2, int depth)`, which generates the points of a single segment of the Koch curve.

i. Create a list `points` of type `List<Point>`.

ii. Check whether the value of `depth` is equal to `0`.

A. If so, add the points `p1` and `p2` to the list.

B. Return the list as the base case of the recursion.

iii. Calculate point `a`, which lies at one third of the segment between `p1` and `p2`.

iv. Calculate point `b`, which lies at two thirds of the segment between `p1` and `p2`.

v. Set the rotation angle `angle` to `Math.PI / 3.0`, that is, `60` degrees.

vi. Calculate the coordinate differences `dx` and `dy` between points `b` and `a`.

vii. Calculate point `peak`, which forms the vertex of the triangular protrusion of the Koch segment.

- Use the formula for rotating a vector by the angle `angle`.
- Use the `Math.cos(...)` and `Math.sin(...)` functions.

viii. Recursively generate four segments: from `p1` to `a`, from `a` to `peak`, from `peak` to `b`, and from `b` to `p2`.

ix. Add the points of the first segment to the result list.

x. Add the points of the following segments while skipping the first point of each of them to avoid duplicates at the joins.

xi. Return the `points` list.

(d) Implement the private method `drawKochSnowflake(Graphics2D g2d)`, which prepares the geometry and draws the Koch snowflake.

i. Calculate the center of the panel as `centerX` and `centerY`.

ii. Calculate the radius `r` as `80%` of half of the smaller panel dimension.

iii. Set the angle offset `angleOffset` to `-Math.PI / 2` to rotate the shape.

iv. Calculate the angular spacing `step` as `2 * Math.PI / 3`.

v. Create an array `vertices` of type `Point[]` with size `3`.

vi. In a loop, calculate the three vertices of the equilateral triangle.

- For each value `i`, calculate the `x` coordinate using `Math.cos(step * i + angleOffset)`.
- Calculate the `y` coordinate using `Math.sin(step * i + angleOffset)`.
- Create a point and store it in the `vertices` array.

vii. Create the `allPoints` list, which will store all points of the snowflake outline.

viii. Generate the three sides of the snowflake by invoking `kochSegment(...)` for pairs of triangle vertices.

ix. Add the points of the first side to the `allPoints` list.

x. Add the points of the second and third sides while skipping the first point to avoid repetitions.

xi. Create a `Path2D.Double` object.

xii. Move the beginning of the path to the first point from the `allPoints` list.

xiii. Iterate over the remaining points and add consecutive segments using `path.lineTo(...)`.

xiv. Close the path by invoking `path.closePath()`.

xv. Set the fill color to `new Color(100, 180, 255)` and fill the path.

xvi. Set the outline color to `new Color(0, 80, 160)`.

xvii. Set the line thickness using `new BasicStroke(1.0f)`.

xviii. Draw the outline of the path by invoking `g2d.draw(path)`.

(e) Override the `paintComponent(Graphics g)` method to handle component drawing.

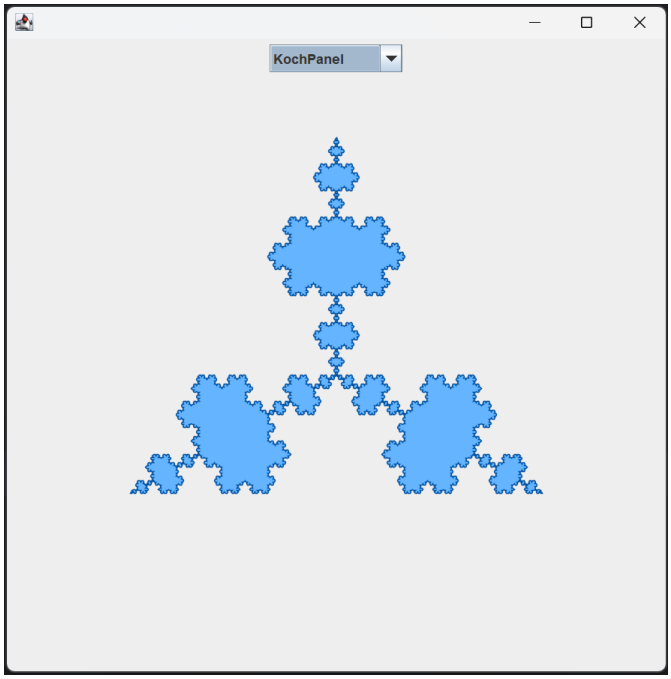
i. Invoke `super.paintComponent(g)`.

ii. Cast the `Graphics` object to `Graphics2D`.

iii. Enable antialiasing by setting `RenderingHints.KEY_ANTIALIASING` to `RenderingHints.VALUE_ANTIALIAS_ON`.

iv. Invoke the `drawKochSnowflake(g2d)` method to draw the Koch snowflake.

25. Check whether the result of drawing the snowflake is similar to the following:



26. ★ Milestone 3: present the result of your work to the instructor